Assignment: Git & GitHub Fundamentals

Part A: Theory Questions

Q1. What problem does Version Control solve in software development?

Answer:

Version Control helps developers manage changes made to source code over time. It solves problems such as:

- Prevents loss of code due to accidental deletion.
- Keeps a history of all changes.
- Allows multiple developers to work on the same project simultaneously.
- Makes it easy to restore previous versions if errors occur.
- Tracks who made each change and when.

Q2. Define Version Control System (VCS).

Answer:

A **Version Control System (VCS)** is a software tool that records and manages changes made to files over time. It enables developers to track modifications, collaborate with others, and restore previous versions whenever needed.

Q3. What is Git?

Answer:

Git is a free, open-source **Distributed Version Control System (DVCS)** created by Linus Torvalds in 2005. It helps developers track code changes, manage different versions of a project, and collaborate efficiently with team members.

Q4. Explain the difference between Git and GitHub.

Answer:

Git	GitHub
Git is a Version Control System.	GitHub is a cloud-based platform for hosting Git repositories.
Works locally on your computer.	Works online over the internet.
Tracks and manages code changes.	Stores repositories and enables collaboration.
Can be used without GitHub.	Uses Git to manage repositories.

Q5. What is a Repository?

Answer:

A **Repository (Repo)** is a storage location that contains a project's files, folders, and complete version history. Repositories can be:

- **Local Repository:** Stored on your computer.
- **Remote Repository:** Stored on platforms like GitHub.

Q6. What is a Commit in Git?

Answer:

A **Commit** is a snapshot of the project at a specific point in time. Every commit saves the changes made to the project along with a commit message describing those changes.

Example:

git commit -m "Added login page"

Q7. What is the purpose of the Staging Area?

Answer:

The **Staging Area** is an intermediate area where changes are prepared before creating a commit. It allows developers to choose which files should be included in the next commit.

Q8. Explain the following Git architecture components.

a) Working Directory

The Working Directory is the folder where developers create, edit, and delete project files.

b) Staging Area

The Staging Area temporarily stores selected changes before they are committed.

c) Local Repository

The Local Repository stores all commits and version history on the developer's computer.

d) Remote Repository

The Remote Repository is an online copy of the project stored on platforms like GitHub, allowing collaboration among team members.

Q9. What is the purpose of the following commands?

a) git init

Initializes a new Git repository in the current project folder.

git init

b) git status

Displays the current status of the repository, including modified, staged, and untracked files.

git status

c) git add .

Adds all modified and new files to the Staging Area.

git add .

d) git commit -m "message"

Creates a new commit with the staged changes and stores them in the Local Repository.

git commit -m "Initial Commit"

e) git push

Uploads local commits to the Remote Repository (GitHub).

git push origin main

Q10. Explain the industry workflow of Git from cloning a project to pushing changes.

Answer:

The standard Git workflow used in the software industry is:

1. Clone the repository from GitHub.

git clone <repository-url>

2. Open the project and make the required changes.

3. Check the status.

git status

4. Add the modified files.

git add .

5. Commit the changes.

git commit -m "Describe your changes"

6. Pull the latest updates from GitHub.

git pull origin main

7. Push the changes to GitHub.

git push origin main

This workflow ensures proper version control and smooth collaboration among developers.

Q11. What information is stored inside a Git commit?

Answer:

A Git commit stores:

- Project snapshot (changed files)
- Commit message
- Author name
- Author email
- Date and time of commit
- Unique Commit ID (SHA Hash)
- Reference to the previous commit

Q12. Why is Git considered a Distributed Version Control System?

Answer:

Git is called a **Distributed Version Control System (DVCS)** because every developer has a complete copy of the repository, including the entire project history. Developers can work offline and synchronize changes with remote repositories later.

Q13. What are the advantages of using GitHub?

Answer:

Advantages of GitHub include:

- Cloud storage for Git repositories.
- Easy collaboration among developers.
- Backup of project code.
- Pull Requests for code review.
- Issue tracking and project management.
- Continuous Integration (CI/CD) support.
- Access from anywhere using the internet.

Q14. What is the difference between Push and Pull?

Answer:

Push	**Pull**
Sends local commits to the remote repository.	Downloads the latest changes from the remote repository.
Uploads changes.	Retrieves changes.
Command: git push	Command: git pull

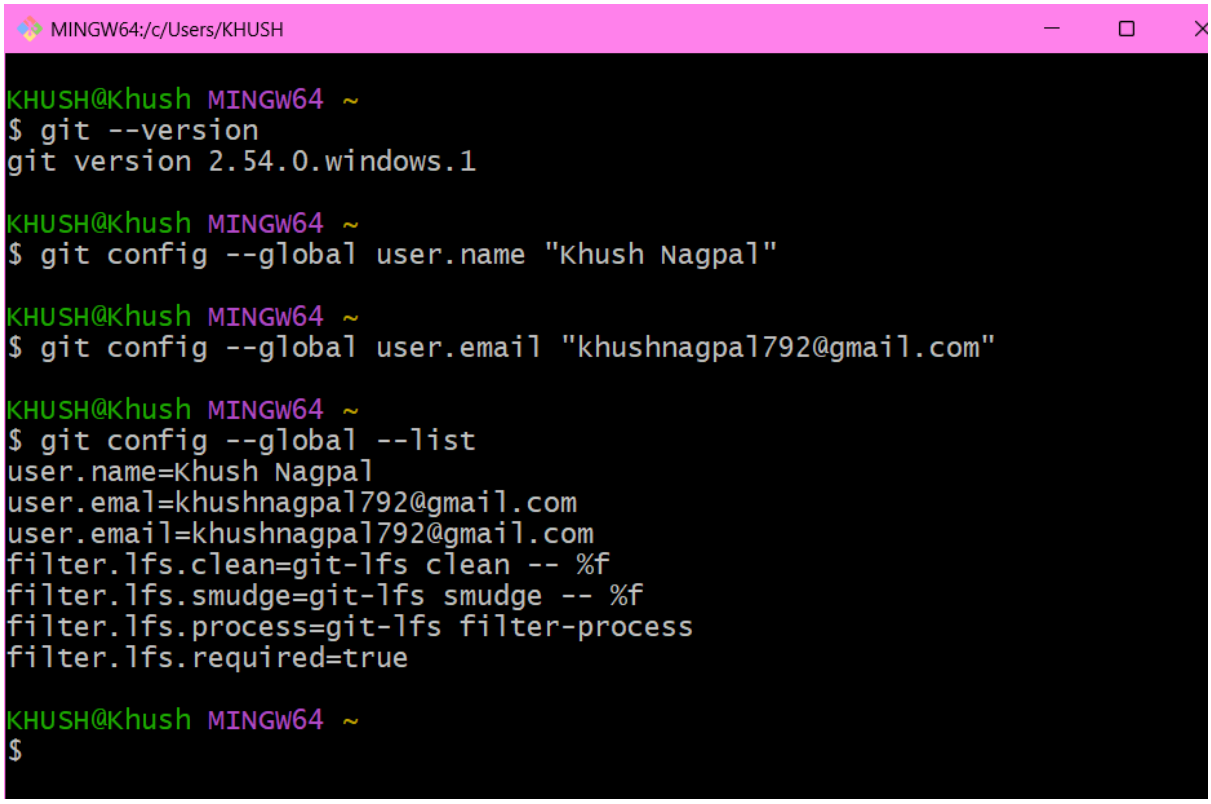
Q15. Explain the role of Git in team collaboration.

Answer:

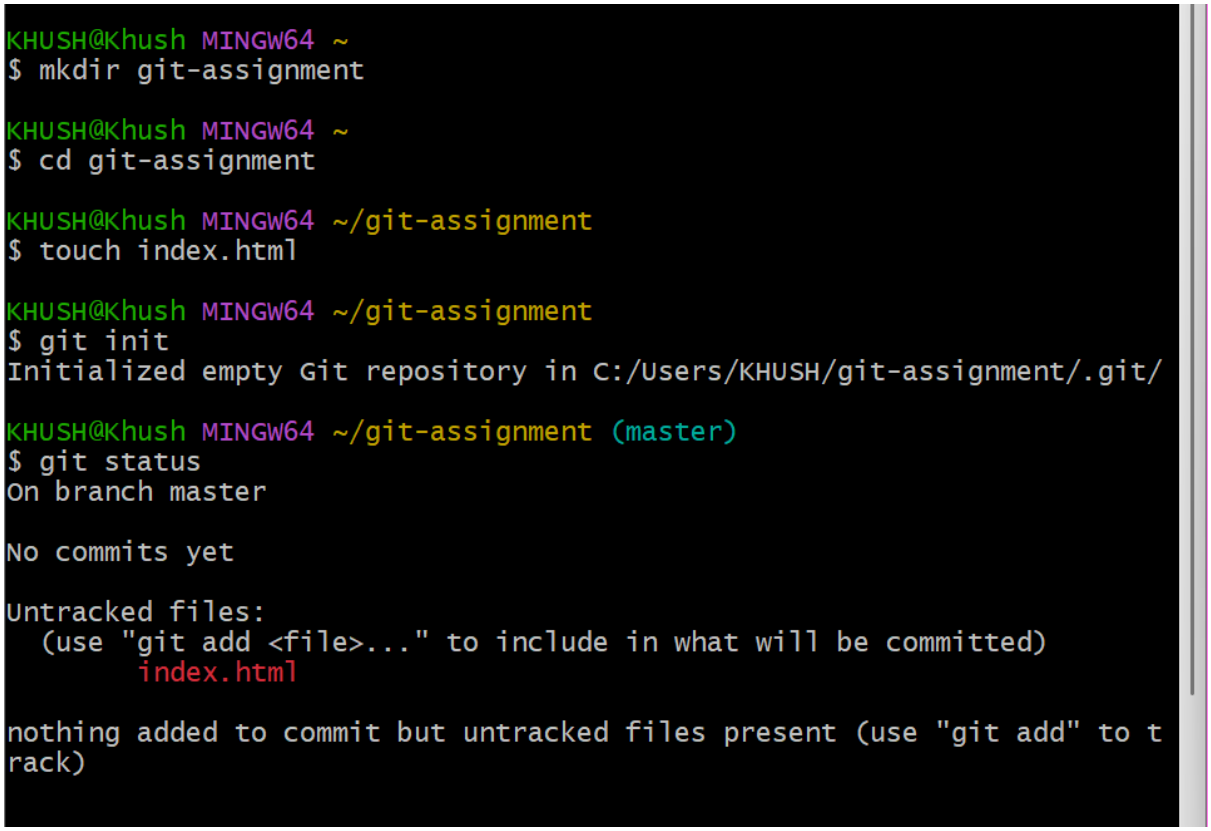
Git plays a vital role in team collaboration by allowing multiple developers to work on the same project efficiently. Each developer can work on separate branches without affecting the main codebase. Git tracks every change, records who made it, and helps merge contributions safely. It also enables teams to review code through pull requests, resolve merge conflicts, and maintain a complete history of the project. This improves coordination, reduces errors, and makes software development faster and more organized.

Part B: Practical Assignment

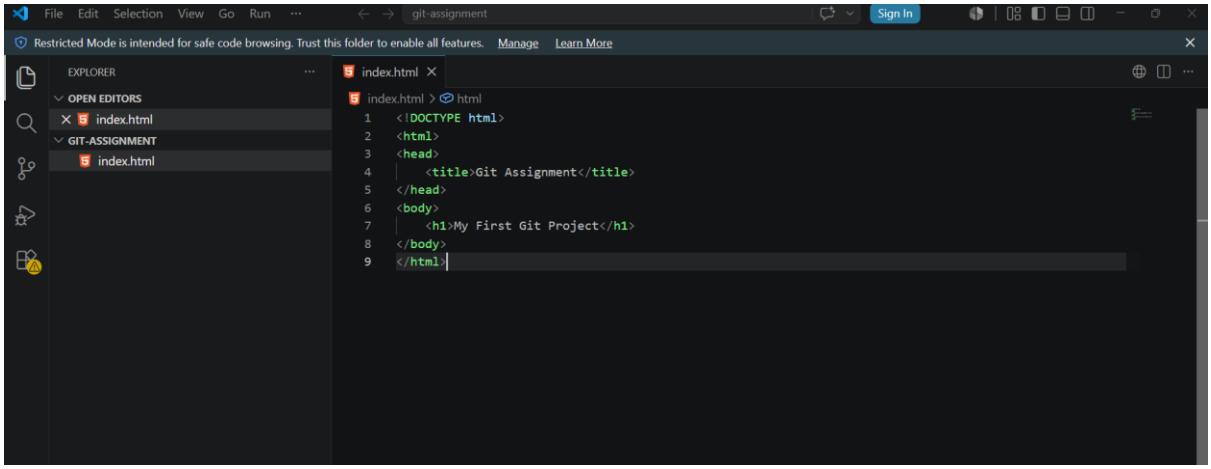
Task 1: Git Configuration

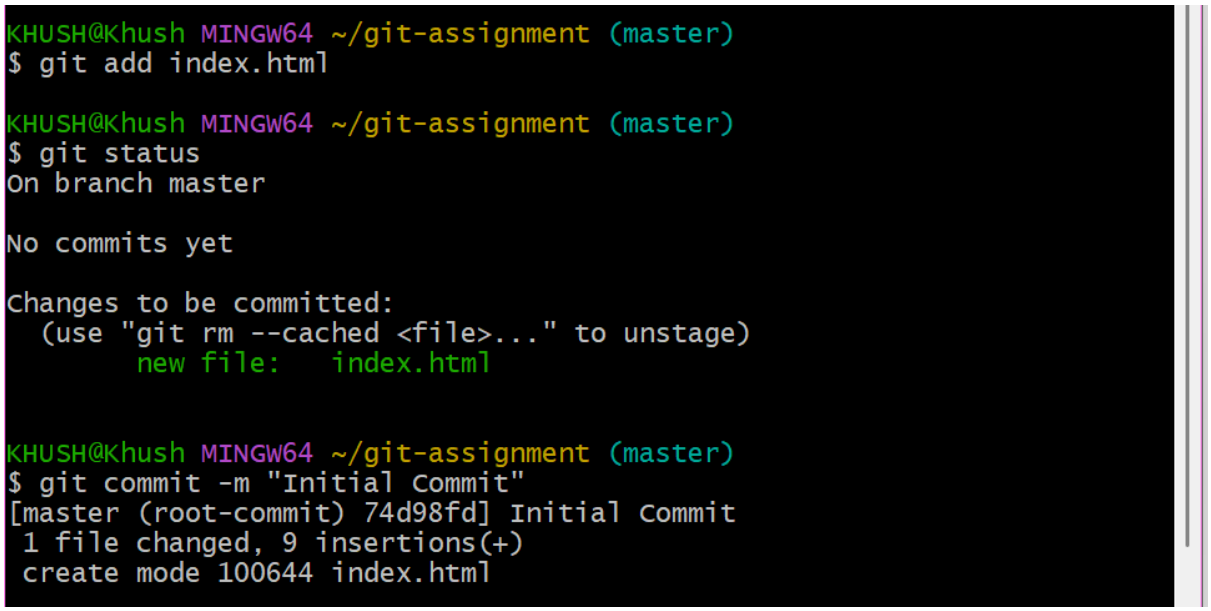


Task 2: Create a Local Repository

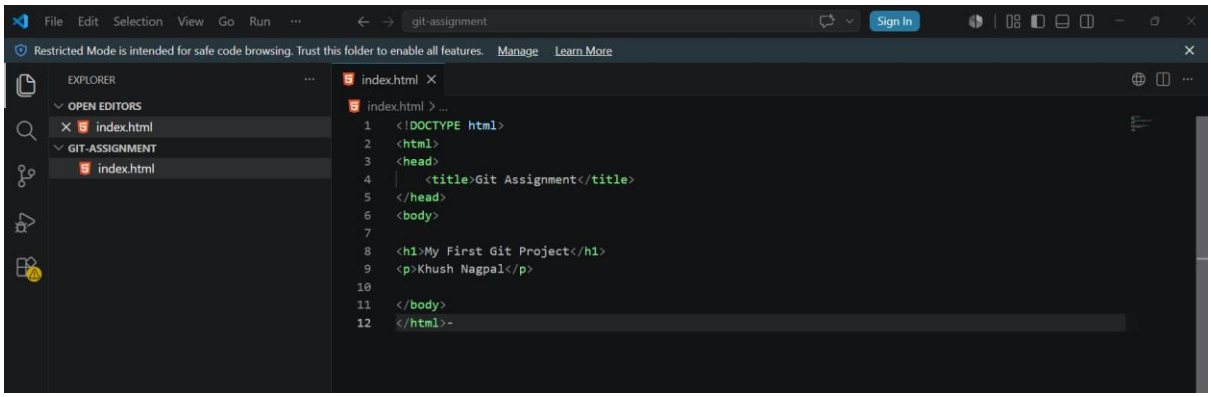


Task 3: First Commit



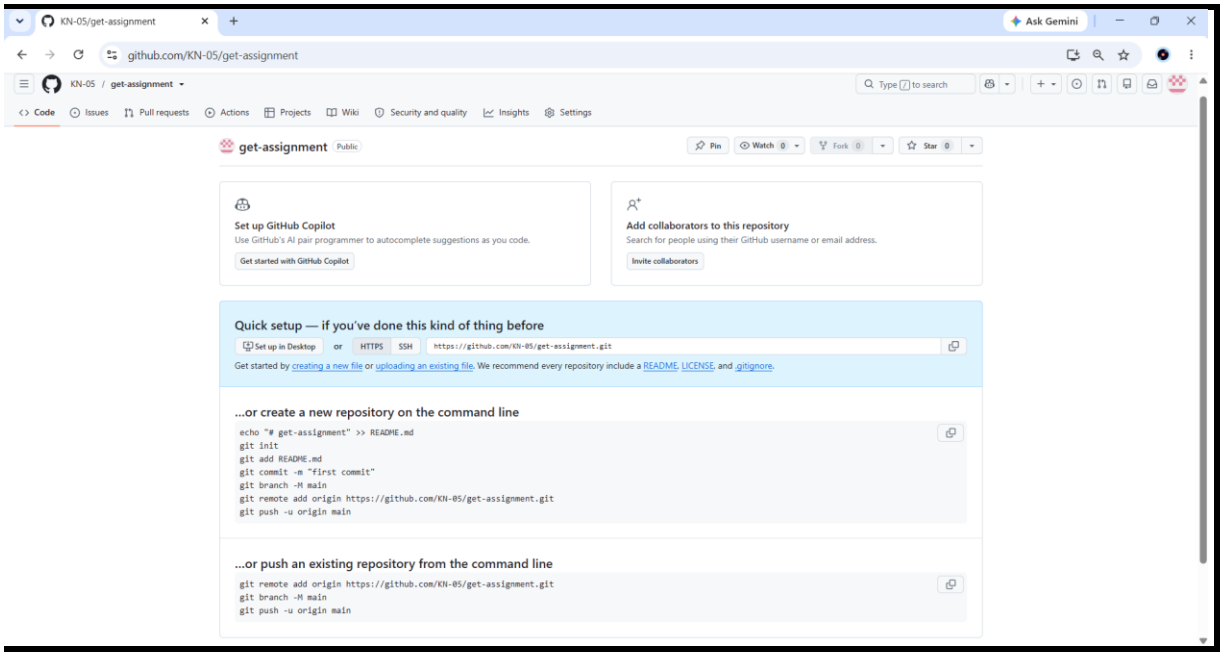


4. View commit history.

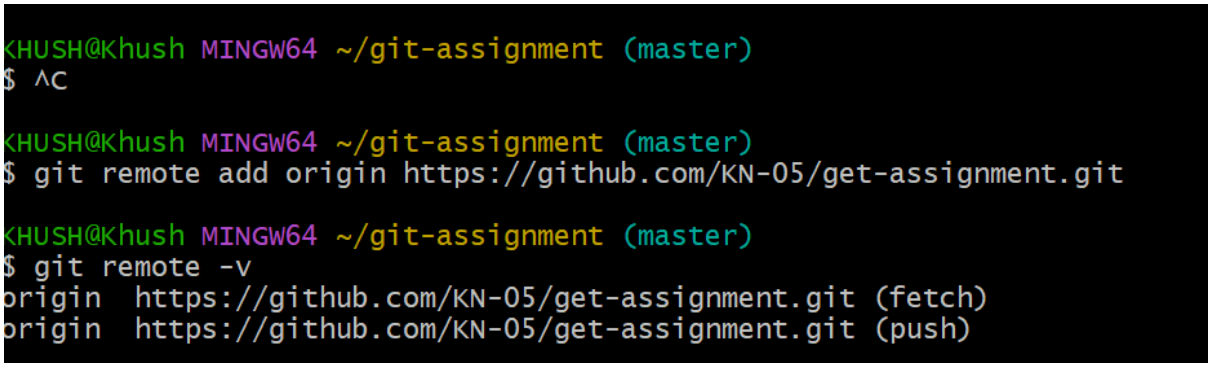



```
KHUSH@Khush MINGW64 ~/git-assignment (master)
$ git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
        add

KHUSH@Khush MINGW64 ~/git-assignment (master)
$ git add index.html

KHUSH@Khush MINGW64 ~/git-assignment (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html


KHUSH@Khush MINGW64 ~/git-assignment (master)
$ git commit -m "Added Student Name"
[master dd0ed77] Added Student Name
 1 file changed, 5 insertions(+), 2 deletions(-)

KHUSH@Khush MINGW64 ~/git-assignment (master)
$ git log
commit dd0ed77e079b69045cef78b231184501aa5951fc (HEAD -> master)
Author: Khush Nagpal <khushnagpal792@gmail.com>
Date:   Wed Jul 1 20:42:12 2026 +0530

    Added Student Name

commit 74d98fde60cb82478e6e2cfe27465a00d25baf22
Author: Khush Nagpal <khushnagpal792@gmail.com>
Date:   Wed Jul 1 20:13:52 2026 +0530

    Initial Commit

KHUSH@Khush MINGW64 ~/git-assignment (master)
```

5. View commit history.



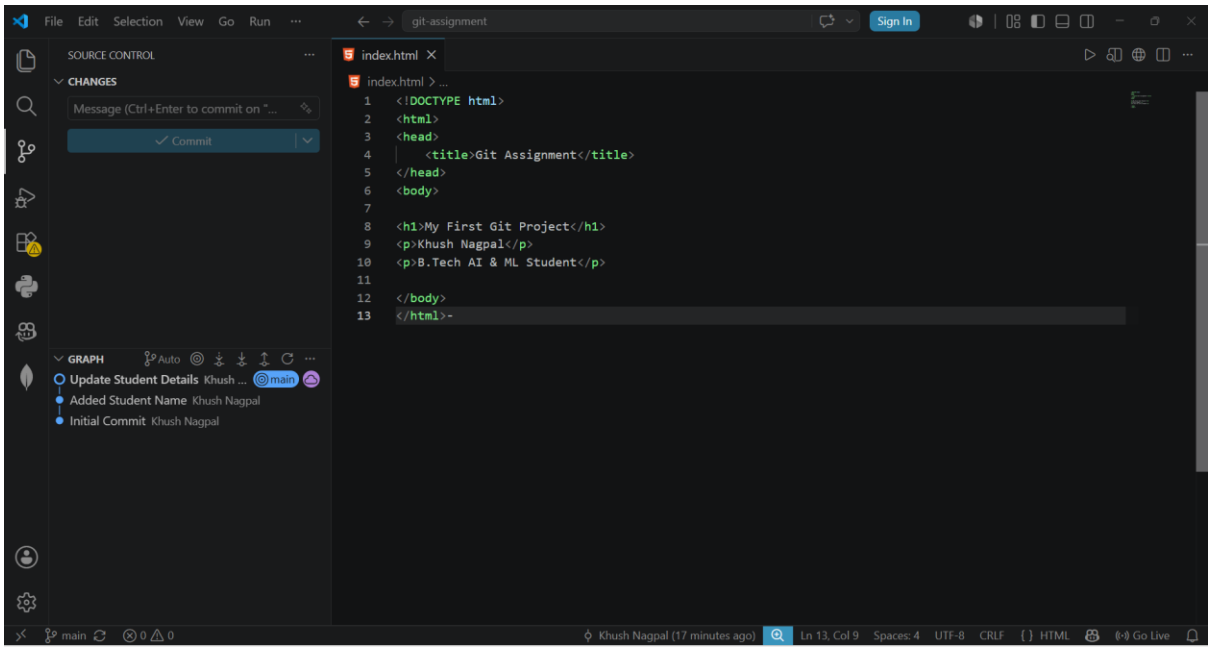
Task 6: Connect Local Repository to GitHub



Task 7: Push Project to GitHub

```
KHUSH@Khush MINGW64 ~/git-assignment (master)
$ git branch -M main

KHUSH@Khush MINGW64 ~/git-assignment (main)
$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 590 bytes | 295.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/KN-05/get-assignment.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Task 8: VS Code Git Operations




```
KHUSH@Khush MINGW64 ~/git-assignment (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

KHUSH@Khush MINGW64 ~/git-assignment (main)
$
```